

COMPUTER PROGRAMMING

Electrical Engineering Department — UET Nowshera

LAB 3

Lab Title: Relational Operators, Logical Operators and Conditional Statements in C++

1. OBJECTIVE

The goal of this lab is to build an understanding of relational operators, logical operators, and conditional statements in C++. Students practise comparing values, evaluating compound conditions, and directing program flow using if, if-else, and else-if constructs.

2. THEORY

By default, a C++ program runs its instructions one after another in sequence. In many real-world situations, however, a program must make decisions based on specific conditions. To support this, C++ provides relational operators, logical operators, and conditional control structures.

Relational Operators

Relational operators are used to compare two values and produce a boolean result — either true or false:

- < — Less than
- > — Greater than
- <= — Less than or equal to
- >= — Greater than or equal to
- == — Equal to
- != — Not equal to

Logical Operators

Logical operators are used to combine or modify multiple conditions:

- `&&` — Logical AND (true if both conditions are true)
- `||` — Logical OR (true if at least one condition is true)
- `!` — Logical NOT (reverses the condition)

Conditional Statements

Conditional statements enable a program to choose between different code paths based on whether a condition evaluates to true or false. The three primary forms are:

- if statement — runs a block of code only when the specified condition holds true.
- if-else statement — runs one block if the condition is true, and a different block if it is false.
- else-if statement — links several conditions together to enable multi-way decision branching.

3. SOFTWARE USED

- DEV-C++ IDE
- C++ Compiler (bundled with DEV-C++)

4. PROGRAMS

Program 1 — Comparison Using Relational Operators

This program accepts two integers from the user and evaluates their relationship using relational operators, printing the corresponding comparison result.

Algorithm:

- Start the program.

- Declare two integer variables.
- Take two numbers as input from the user.
- Compare the numbers using relational operators.
- Display the comparison result.
- End the program.

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;

    // Compare using relational operators
    if(a == b)
        cout << "Both numbers are equal";
    if(a > b)
        cout << "First number is greater";
    if(a < b)
        cout << "First number is smaller";

    return 0;
}
```

Program 2 — Range Check Using Logical Operators

This program determines whether a user-provided number lies within the range of 0 to 100 by applying the logical AND (&&) operator.

Algorithm:

- Start the program.
- Declare an integer variable.
- Take a number as input from the user.
- Check if the number is greater than 0 AND less than 100.
- Display the appropriate result.
- End the program.

```
#include <iostream>
using namespace std;

int main()
```

```
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    // Use logical AND to check both conditions simultaneously
    if(num > 0 && num < 100)
        cout << "Number is between 0 and 100";
    else
        cout << "Number is not between 0 and 100";

    return 0;
}
```

Program 3 — Positive Number Check Using if Statement

This program takes an integer as input and uses a straightforward if statement to decide whether the value is positive.

Algorithm:

- Start the program.
- Declare an integer variable.
- Take a number as input from the user.
- Check whether the number is positive using an if statement.
- Display the result.
- End the program.

```
#include <iostream>
using namespace std;

int main()
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    // if statement checks a single condition
    if(num > 0)
        cout << "The number is positive";

    return 0;
}
```

5. CONCLUSION

This lab provided practical experience with relational and logical operators in C++. The if, if-else, and else-if conditional statements were implemented to manage program flow according to varying conditions. A solid grasp of these concepts is indispensable for writing programs that can make intelligent decisions.